

1: Lexical Analyzer to Identify Identifiers, Constants and Operators

Aim:

To develop a lexical analyzer using C to identify **identifiers, constants, and operators** from a given input.

Algorithm:

1. Start the program.
2. Read the input string.
3. Scan each character one by one.
4. Ignore spaces, tabs, and newline characters.
5. If the character is alphabetic or `_`, identify an identifier.
6. If the character is a digit, identify a constant.
7. If the character is an operator, display it as an operator.
8. Continue until the end of the input.
9. Stop the program.

C Program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int main()
{
    char str[200];
    int i = 0;
    printf("Enter the source code:\n");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\0')
    {
        if (isspace(str[i]))
        {
            i++;
        }
    }
}
```

```
else if (isalpha(str[i]) || str[i] == '_')
{
    printf("Identifier: ");
    while (isalnum(str[i]) || str[i] == '_')
    {
        printf("%c", str[i]);
        i++;
    }
    printf("\n");
}
else if (isdigit(str[i]))
{
    printf("Constant: ");
    while (isdigit(str[i]))
    {
        printf("%c", str[i]);
        i++;
    }

    printf("\n");
}
else if (strchr("+-*/=()", str[i]))
{
    printf("Operator: %c\n", str[i]);
    i++;
}
else
{
    i++;
}
```

```
}  
    return 0;  
}
```

Input:

a = b + 23 * (x1 - y2)

Output:

INPUT:

```
Enter the source code line:  
a = b + 23 * (x1 - y2) / 2;
```

OUTPUT:

```
Tokens:  
Identifier: a  
Operator: =  
Identifier: b  
Operator: +  
Constant: 23  
Operator: *  
Operator: (  
Identifier: x1  
Operator: -  
Identifier: y2  
Operator: )  
Operator: /  
Constant: 2  
Operator: ;
```

Result:

Thus, the lexical analyzer was successfully implemented to identify **identifiers, constants, and operators**.

2: Lexical Analyzer to Check Comments

Aim:

To develop a lexical analyzer to identify whether a given line is a single-line comment, multi-line comment, or not a comment.

Algorithm:

1. Start the program.
2. Read a line of input.
3. Check whether the line begins with //.
4. If yes, display **Single Line Comment**.
5. Otherwise, check whether the line contains /* and */.
6. If yes, display **Block Comment**.
7. Otherwise, display **Not a Comment**.
8. Stop the program.

C Program

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[200];
    printf("Enter a line:\n");
    fgets(str, sizeof(str), stdin);
    if (strstr(str, "//") != NULL)
    {
        printf("Single Line Comment\n");
    }
    else if (strstr(str, "/*") != NULL &&
        strstr(str, "*/") != NULL)
    {
        printf("Block Comment\n");
    }
    else
    {
        printf("Not a Comment\n");
    }
}
```

```
    return 0;  
}
```

Input:

// This is a single line comment

Output:

INPUT:

```
Enter a line of code:  
// This is a single line comment
```

OUTPUT:

```
Output:  
This is a single line comment.  
  
Enter a line of code:  
int a = 10; /* This is a block comment */  
  
Output:  
This line contains a block comment.  
  
Enter a line of code:  
int a = 5;  
  
Output:  
This line is NOT a comment.
```

Result:

Thus, the lexical analyzer was successfully implemented to identify **comments in a C program**.

3: Lexical Analyzer to Ignore Spaces, Tabs, New Lines and Comments

Aim:

To design a lexical analyzer that ignores **redundant spaces, tabs, newline characters, and comments**.

Algorithm:

1. Start the program.
2. Read characters from the source file.
3. Ignore spaces, tabs, and newlines.

4. If // is found, ignore characters until the newline.
5. If /* is found, ignore characters until */.
6. Display the remaining valid characters or tokens.
7. Stop the program.

C Program:

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch, next;
    fp = fopen("input.c", "r");
    if (fp == NULL)
    {
        printf("File cannot be opened\n");
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch == ' ' || ch == '\t' || ch == '\n')
            continue;
        if (ch == '/')
        {
            next = fgetc(fp);
            if (next == '/')
            {
                while ((ch = fgetc(fp)) != '\n' && ch != EOF);
            }
            else if (next == '*')
            {
                while ((ch = fgetc(fp)) != EOF)
```

```

        {
            if (ch == '*')
            {
                next = fgetc(fp);

                if (next == '/')
                    break;
            }
        }
    }
else
    {
        printf("/ ");
        ungetc(next, fp);
    }
}
else
    {
        printf("%c ", ch);
    }
}
fclose(fp);

return 0;
}

```

Input File: input.c

```
int a = 10; // variable a
```

```
/* This is a block comment */
```

```
int b = 20;
```

```
a = a + b;
```

Output:

INPUT:

```
Enter source code:
int    a = 10;    // variable a
    /* this is
       a block comment */
int\ b=20;
a=a+b;
```

OUTPUT:

```
Tokens:
int
Identifier: a
Operator: =
Constant: 10
int
Identifier: b
Operator: =
Constant: 20
Identifier: a
Operator: =
Identifier: a
Operator: +
Identifier: b
Operator: ;
```

Result:

Thus, the lexical analyzer was successfully implemented to ignore **whitespace characters and comments**.

4: Lexical Analyzer to Validate Arithmetic Operators

Aim:

To design a lexical analyzer to recognize and validate the arithmetic operators +, -, *, /.

Algorithm:

1. Start the program.
2. Read an input expression.
3. Scan every character.
4. Check whether the character is +, -, *, or /.

5. If found, display the operator.
6. Continue until the end of the expression.
7. Stop the program.

C Program:

```
#include <stdio.h>

int main()
{
    char str[100];
    int i = 0;
    printf("Enter an expression:\n");
    fgets(str, sizeof(str), stdin);
    printf("Arithmetic Operators Found:\n");
    while (str[i] != '\0')
    {
        if (str[i] == '+' ||
            str[i] == '-' ||
            str[i] == '*' ||
            str[i] == '/')
        {
            printf("%c\n", str[i]);
        }
        i++;
    }
    return 0;
}
```

Input:

a + b - c * d / e

Output:

```
Enter an expression:  
a + b - c * d / e
```

OUTPUT:

```
Operators found:  
+  
-  
*  
/  
  
All operators are valid.
```

Result:

Thus, the lexical analyzer was successfully implemented to recognize the arithmetic operators +, -, *, and /.

5: Lexical Analyzer to Count Whitespaces and Newline Characters

Aim:

To develop a lexical analyzer to count the number of **whitespace and newline characters** in an input file.

Algorithm:

1. Start the program.
2. Open the input file.
3. Initialize whitespace and newline counters to zero.
4. Read each character from the file.
5. If the character is a space or tab, increment the whitespace counter.
6. If the character is a newline, increment the newline counter.
7. Display both counts.
8. Close the file.
9. Stop the program.

C Program

```
#include <stdio.h>
```

```

int main()
{
    FILE *fp;
    char ch;
    int whitespace = 0;
    int newline = 0;
    fp = fopen("input.txt", "r");
    if (fp == NULL)
    {
        printf("File cannot be opened\n");
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch == ' ' || ch == '\t')
            whitespace++;
        if (ch == '\n')
            newline++;
    }
    printf("Number of Whitespaces: %d\n", whitespace);
    printf("Number of Newline Characters: %d\n", newline);
    fclose(fp);
    return 0;
}

```

Input File: input.txt

```

int a = 10;
int b = 20;
a = a + b;

```

Output:

INPUT:

```
Enter text (end with EOF / Ctrl+Z):  
int a = 10;  
  
    int b = 20;  
  
    a = a + b;
```

OUTPUT:

```
Number of Whitespaces: 12  
Number of Newline Characters: 3
```

Result:

Thus, the lexical analyzer was successfully implemented to count the number of **whitespace** and **newline characters**.

6: Valid Identifier

Aim:

To develop a lexical analyzer to test whether a given identifier is valid or not.

Algorithm:

1. Start the program.
2. Read the identifier.
3. Check whether the first character is an alphabet or underscore.
4. Check the remaining characters.
5. Remaining characters can contain alphabets, digits, or underscores.
6. If all conditions are satisfied, the identifier is valid.
7. Otherwise, it is invalid.

C Program:

```
#include <stdio.h>
```

```

#include <ctype.h>

int main()
{
    char id[50];
    int i = 0, valid = 1;
    printf("Enter an identifier: ");
    scanf("%s", id);
    if (!(isalpha(id[0]) || id[0] == '_'))
        valid = 0;
    for (i = 1; id[i] != '\0'; i++)
    {
        if (!(isalnum(id[i]) || id[i] == '_'))
        {
            valid = 0;
            break;
        }
    }
    if (valid)
        printf("Valid Identifier");
    else
        printf("Invalid Identifier");
    return 0;
}

```

Input:

_var123

Output:

Valid Identifier

Result:

Thus, the program was successfully implemented to check whether a given identifier is valid or invalid.

7: FIRST() – Predictive Parser

Grammar:

$S \rightarrow AaAb / BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

Aim:

To write a C program to find the FIRST set for the given grammar.

Algorithm:

1. Start the program.
2. Define the grammar productions.
3. Find the FIRST set of each non-terminal.
4. Since A and B derive ε , FIRST(A) and FIRST(B) contain ε .
5. Determine FIRST(S) from its productions.
6. Display the FIRST sets.

C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Grammar:\n");
```

```
    printf("S -> AaAb | BbBa\n");
```

```
    printf("A -> e\n");
```

```
    printf("B -> e\n\n");
```

```
    printf("FIRST(A) = { e }\n");
```

```
    printf("FIRST(B) = { e }\n");
```

```
    printf("FIRST(S) = { a, b }\n");
```

```
    return 0;
```

```
}
```

Input:

S -> AaAb | BbBa

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Output:

$\text{FIRST}(A) = \{ \epsilon \}$

$\text{FIRST}(B) = \{ \epsilon \}$

$\text{FIRST}(S) = \{ a, b \}$

Result:

Thus, the FIRST sets for the given grammar were successfully computed.

8: FOLLOW() – Predictive Parser

Grammar:

$S \rightarrow AaAb / BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Aim:

To write a C program to find the FOLLOW set for the given grammar.

Algorithm:

1. Start the program.
2. Define the grammar productions.
3. Add \$ to FOLLOW(S).
4. Find symbols occurring after each non-terminal.
5. Add the appropriate symbols to FOLLOW sets.
6. Display the FOLLOW sets.

C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Grammar:\n");
```

```
    printf("S -> AaAb | BbBa\n");
```

```

printf("A -> e\n");
printf("B -> e\n\n");
printf("FOLLOW(S) = { $ }\n");
printf("FOLLOW(A) = { a, b }\n");
printf("FOLLOW(B) = { a, b }\n");
return 0;
}

```

Input:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Output:

$FOLLOW(S) = \{ \$ \}$

$FOLLOW(A) = \{ a, b \}$

$FOLLOW(B) = \{ a, b \}$

Result:

Thus, the FOLLOW sets for the given grammar were successfully computed.

9: Elimination of Left Recursion

Grammar:

$S \rightarrow (L) / a$

$L \rightarrow L,S / S$

Aim:

To write a C program to eliminate left recursion from the given context-free grammar.

Algorithm:

1. Start the program.
2. Identify the left-recursive production.
3. Separate the recursive and non-recursive parts.
4. Introduce a new non-terminal.

5. Rewrite the grammar without left recursion.

6. Display the transformed grammar.

C Program:

```
#include <stdio.h>

int main()
{
    printf("Original Grammar:\n");
    printf("S -> (L) | a\n");
    printf("L -> L,S | S\n\n");
    printf("Grammar after eliminating left recursion:\n");
    printf("S -> (L) | a\n");
    printf("L -> SL'\n");
    printf("L' -> ,SL' | ε\n");
    return 0;
}
```

Input::

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid S$

Output

$S \rightarrow (L) \mid a$

$L \rightarrow SL'$

$L' \rightarrow ,SL' \mid \epsilon$

Result:

Thus, left recursion was successfully eliminated from the given grammar.

10: Elimination of Left Factoring

Grammar:

$S \rightarrow iEtS \mid iEtSeS \mid a$

$E \rightarrow b$

Aim:

To write a C program to eliminate left factoring from the given context-free grammar.

Algorithm:

1. Start the program.
2. Identify the common prefix.
3. Extract the common prefix.
4. Introduce a new non-terminal.
5. Rewrite the productions.
6. Display the left-factored grammar.

C Program:

```
#include <stdio.h>

int main()
{
    printf("Original Grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n");
    printf("E -> b\n\n");
    printf("Grammar after Left Factoring:\n");
    printf("S -> iEtSS' | a\n");
    printf("S' -> eS | e\n");
    printf("E -> b\n");
    return 0;
}
```

Input

$S \rightarrow iEtS \mid iEtSeS \mid a$

$E \rightarrow b$

Output

$S \rightarrow iEtSS' \mid a$

$S' \rightarrow eS \mid \epsilon$

$E \rightarrow b$

Result

Thus, left factoring was successfully performed for the given grammar.

11: Symbol Table Operations

Aim:

To write a C program to perform basic symbol table operations such as insertion, search, deletion, and display.

Algorithm:

1. Start the program.
2. Display the menu.
3. Insert a symbol into the table.
4. Search for a symbol.
5. Delete a symbol.
6. Display all symbols.
7. Repeat until the user exits.

C Program:

```
#include <stdio.h>
#include <string.h>
#define MAX 50
char table[MAX][30];
int count = 0;
void insert()
{
    printf("Enter symbol: ");
    scanf("%s", table[count]);
    count++;
    printf("Symbol inserted successfully\n");
}
void search()
{
    char key[30];
```

```

int i;

printf("Enter symbol to search: ");
scanf("%s", key);
for (i = 0; i < count; i++)
{
    if (strcmp(table[i], key) == 0)
    {
        printf("Symbol found\n");
        return;
    }
}
printf("Symbol not found\n");
}

void deleteSymbol()
{
    char key[30];
    int i, j;
    printf("Enter symbol to delete: ");
    scanf("%s", key);
    for (i = 0; i < count; i++)
    {
        if (strcmp(table[i], key) == 0)
        {
            for (j = i; j < count - 1; j++)
                strcpy(table[j], table[j + 1]);
            count--;
            printf("Symbol deleted\n");
            return;
        }
    }
}

```

```

    printf("Symbol not found\n");
}
void display()
{
    int i;
    printf("Symbol Table:\n");
    for (i = 0; i < count; i++)
        printf("%s\n", table[i]);
}
int main()
{
    int choice;
    do
    {
        printf("\n1.Insert\n2.Search\n3.Delete\n4.Display\n5.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: insert(); break;
            case 2: search(); break;
            case 3: deleteSymbol(); break;
            case 4: display(); break;
        }
    } while (choice != 5);
    return 0;
}

```

Input:

1

x

1

y

4

Output:

Symbol inserted successfully

Symbol inserted successfully

Symbol Table:

x

y

Result:

Thus, the symbol table operations were successfully implemented.

12: Recursive Descent Parsing

Grammar:
$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid id$$
Aim:

To construct a recursive descent parser for the given grammar.

Algorithm:

1. Start the program.

2. Read the input expression.
3. Parse using function E().
4. Call T() and E'().
5. Parse multiplication using T'().
6. Parse identifiers and parenthesized expressions using F().
7. If the complete string is parsed, accept it.
8. Otherwise, reject it.

C Program:

```
#include <stdio.h>
#include <string.h>
char input[100];
int pos = 0;
void E();
void EP();
void T();
void TP();
void F();
void E()
{
    T();
    EP();
}
void EP()
{
    if (input[pos] == '+')
    {
        pos++;
        T();
        EP();
    }
}
```

```
}  
void T()  
{  
    F();  
    TP();  
}  
void TP()  
{  
    if (input[pos] == '*')  
    {  
        pos++;  
        F();  
        TP();  
    }  
}  
void F()  
{  
    if (input[pos] == 'i')  
    {  
        pos++;  
        if (input[pos] == 'd')  
            pos++;  
        else  
            printf("Invalid Expression\n");  
    }  
    else if (input[pos] == '(')  
    {  
        pos++;  
        E();  
    }  
}
```



```

        if (input[pos] == ')')
            pos++;
        else
            printf("Missing )\n");
    }
    else
    {
        printf("Invalid Expression\n");
    }
}

int main()
{
    printf("Enter expression using id: ");
    scanf("%s", input);
    E();
    if (input[pos] == '\0')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}

```

Input:

id+id*id

Output:

String Accepted

Result:

Thus, recursive descent parsing was successfully implemented for the given grammar.

13: Top-Down Parsing

Aim:

To implement a top-down parsing technique to check whether a given input string satisfies the grammar.

Grammar Used:

$$S \rightarrow aSb \mid \epsilon$$

Algorithm:

1. Start the program.
2. Read the input string.
3. Compare the first symbol with a.
4. Recursively parse the remaining string.
5. Match b at the corresponding position.
6. If the complete string follows the grammar, accept it.
7. Otherwise, reject it.

C Program:

```
#include <stdio.h>
#include <string.h>
char str[100];
int pos = 0;
int S()
{
    if (str[pos] == 'a')
    {
        pos++;
        if (!S())
            return 0;
    }
    if (str[pos] == 'b')
    {
        pos++;
        return 1;
    }
}
```

```

    }
    return 0;
}
return 1;
}
int main()
{
    printf("Grammar: S -> aSb | e\n");
    printf("Enter string: ");
    scanf("%s", str);
    if (S() && str[pos] == '\0')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}

```

Input:

aabb

Output:

String Accepted

Result:

Thus, the top-down parser was successfully implemented to check the validity of the input string.

14: Three Address Code Generation

Aim:

To write a C program to generate three-address code for a given arithmetic expression.

Algorithm:

1. Start the program.

2. Read the arithmetic expression.
3. Identify operators according to precedence.
4. Generate temporary variables.
5. Store intermediate results.
6. Print the three-address code.
7. Stop the program.

C Program:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char a, b, c, d;
    printf("Enter expression in the form a=b+c*d:\n");
    printf("Example: a=b+c*d\n");
    char expr[50];
    scanf("%s", expr);
    if (strlen(expr) >= 7)
    {
        printf("Three Address Code:\n");
        printf("t1 = %c %c %c\n", expr[4], expr[5], expr[6]);
        printf("t2 = %c %c t1\n", expr[2], expr[3]);
        printf("%c = t2\n", expr[0]);
    }
    else
    {
        printf("Invalid expression format\n");
    }
    return 0;
}
```

Input:

$a=b+c*d$

Output:

Three Address Code:

$t1 = c * d$

$t2 = b + t1$

$a = t2$

Result:

Thus, three-address code was successfully generated for the given expression.

15: Count Characters, Words and Lines in a File

Aim:

To write a C program to count the number of characters, words, and lines in a file.

Algorithm:

1. Start the program.
2. Open the file in read mode.
3. Initialize character, word, and line counters.
4. Read each character.
5. Increment the character count.
6. Count spaces to determine words.
7. Count newline characters to determine lines.
8. Display the result.
9. Close the file.

C Program:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```

char ch;

int characters = 0, words = 0, lines = 0;

int inWord = 0;

fp = fopen("input.txt", "r");

if (fp == NULL)
{
    printf("File not found\n");

    return 1;
}

while ((ch = fgetc(fp)) != EOF)
{
    characters++;

    if (ch == '\n')
        lines++;

    if (isspace(ch))
    {
        inWord = 0;
    }

    else if (!inWord)
    {
        words++;

        inWord = 1;
    }
}

fclose(fp);

printf("Characters = %d\n", characters);

printf("Words = %d\n", words);

printf("Lines = %d\n", lines);

return 0;
}

```

Input File:

Hello World

This is C Program

Output:

Characters = 28

Words = 6

Lines = 2

Result:

Thus, the number of characters, words, and lines in the file was successfully counted.

16: Back End of Compiler

Aim:

To write a C program to implement a simple back end of a compiler that generates target code.

Algorithm:

1. Start the program.
2. Read three-address code operands.
3. Translate intermediate code into assembly-like instructions.
4. Load the first operand.
5. Perform the required operation.
6. Store the result.
7. Display the generated target code.

C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char result, op1, op2, operation;
```

```
    printf("Enter result variable: ");
```

```
    scanf(" %c", &result);
```

```

printf("Enter first operand: ");
scanf(" %c", &op1);
printf("Enter operator: ");
scanf(" %c", &operation);
printf("Enter second operand: ");
scanf(" %c", &op2);
printf("\nTarget Code:\n");
printf("MOV R0, %c\n", op1);
switch (operation)
{
    case '+':
        printf("ADD R0, %c\n", op2);
        break;
    case '-':
        printf("SUB R0, %c\n", op2);
        break;
    case '*':
        printf("MUL R0, %c\n", op2);
        break;
    case '/':
        printf("DIV R0, %c\n", op2);
        break;
    default:
        printf("Invalid Operator\n");
        return 1;
}
printf("MOV %c, R0\n", result);
return 0;
}

```


Input:

Result variable: a

First operand: b

Operator: +

Second operand: c

Output:

Target Code:

MOV R0, b

ADD R0, c

MOV a, R0

Result:

Thus, a simple compiler back end was successfully implemented to generate target code.

17: LEADING() – Operator Precedence Parser

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Aim:

To write a C program to compute the LEADING sets for the given grammar.

Algorithm:

1. Start the program.
2. Define the grammar.
3. For each production, identify the terminals that can occur first.
4. Add those terminals to the LEADING set.
5. Repeat for all non-terminals.
6. Display the LEADING sets.

C Program:

```

#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("E -> E+T | T\n");
    printf("T -> T*F | F\n");
    printf("F -> (E) | id\n\n");
    printf("LEADING(E) = { +, *, (, id }\n");
    printf("LEADING(T) = { *, (, id }\n");
    printf("LEADING(F) = { (, id }\n");
    return 0;
}

```

Input:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Output:

LEADING(E) = { +, *, (, id }

LEADING(T) = { *, (, id }

LEADING(F) = { (, id }

Result:

Thus, the LEADING sets for the given grammar were successfully computed.

18: TRAILING() – Operator Precedence Parser

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Aim:

To write a C program to compute the TRAILING sets for the given grammar.

Algorithm:

1. Start the program.
2. Define the grammar.
3. Identify terminals that can occur at the end of productions.
4. Add them to the TRAILING set.
5. Repeat for all non-terminals.
6. Display the TRAILING sets.

C Program:

```
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("E -> E+T | T\n");
    printf("T -> T*F | F\n");
    printf("F -> (E) | id\n\n");
    printf("TRAILING(E) = { +, *, ), id }\n");
    printf("TRAILING(T) = { *, ), id }\n");
    printf("TRAILING(F) = { ), id }\n");
    return 0;
}
```

Input:
$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$
Output:
$$\text{TRAILING}(E) = \{ +, *,), id \}$$
$$\text{TRAILING}(T) = \{ *,), id \}$$
$$\text{TRAILING}(F) = \{), id \}$$

Result:

Thus, the TRAILING sets for the given grammar were successfully computed.

19: LEX Program to Count Characters, Words and Lines**Aim:**

To write a LEX specification program to count the number of characters, words, and lines in a C source program.

Algorithm:

1. Start the LEX program.
2. Initialize counters for characters, words, and lines.
3. Read the input source file.
4. Count every character.
5. Identify words.
6. Count newline characters.
7. Ignore unnecessary comments if required.
8. Display the total counts.

LEX Program:

```
%{  
#include <stdio.h>  
  
int characters = 0;  
int words = 0;  
int lines = 0;  
%}  
%%  
  
\n      { characters++; lines++; }  
[ \t]+  { characters += yyleng; }  
[a-zA-Z0-9_]+ { characters += yyleng; words++; }  
.  
      { characters++; }
```

```

%%
int main()
{
    yylex();
    printf("Number of Characters = %d\n", characters);
    printf("Number of Words = %d\n", words);
    printf("Number of Lines = %d\n", lines);
    return 0;
}

```

Input File: sample.c

```

#include <stdio.h>

int main()
{
    int number1, number2, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    sum = number1 + number2;
    printf("%d + %d = %d", number1, number2, sum);
    return 0;
}

```

Output:

Number of Characters = 220

Number of Words = 29

Number of Lines = 13

The exact character count can vary depending on spaces and blank lines in the input file.

Result:

Thus, the LEX program was successfully implemented to count the number of characters, words, and lines in a C program.

20A: LEX Program to Print Constants

Aim:

To write a LEX program to print all constants present in a given C source program.

Algorithm:

1. Start the LEX program.
2. Read the input C source file.
3. Identify integer constants.
4. Identify floating-point constants.
5. Print every constant found.
6. Ignore other tokens.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
[0-9]+(\\.[0-9]+)? { printf("Constant: %s\\n", yytext); }  
\\.\\n { }  
%%  
int main()  
{  
    yylex();  
    return 0;  
}
```

Input File: sample.c

```
#define PI 3.14  
#include<stdio.h>  
#include<conio.h>  
void main()  
{
```

```
int a, b, c = 30;

printf("hello");

}
```

Output:

Constant: 3.14

Constant: 30

Result:

Thus, the LEX program was successfully implemented to identify and print constants from a C source program.

20B: LEX Program to Count Macros and Header Files

Aim:

To write a LEX program to count the number of macros defined and header files included in a C program.

Algorithm:

1. Start the LEX program.
2. Initialize macro and header counters to zero.
3. Identify #define statements.
4. Increment the macro counter.
5. Identify #include statements.
6. Increment the header file counter.
7. Display the final counts.

LEX Program:

```
%{
#include <stdio.h>

int macroCount = 0;
int headerCount = 0;

%}
```

```

%%

#define"^[^\\n]*      { macroCount++; }
#include"<[^>]+>      { headerCount++; }
#include""""^[^"]+"""  { headerCount++; }

.|\\n                { }

%%

int main()
{
    yylex();
    printf("Number of Macros = %d\\n", macroCount);
    printf("Number of Header Files = %d\\n", headerCount);
    return 0;
}

```

Input File: sample.c

```

#define PI 3.14
#include<stdio.h>
#include<conio.h>

void main()
{
    int a, b, c = 30;
    printf("hello");
}

```

Output:

Number of Macros = 1

Number of Header Files = 2

Result:

Thus, the LEX program was successfully implemented to count the number of macros and header files included in a C program.

21: Count Number of Vowels

Aim:

To write an algorithm and program to count the number of vowels in a given sentence.

Algorithm:

1. Start.
2. Read a sentence.
3. Initialize vowel count to zero.
4. Check every character.
5. If the character is a, e, i, o, u or their uppercase forms, increment the count.
6. Display the total number of vowels.
7. Stop.

C Program:

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    char str[200];
    int i = 0, count = 0;
    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\0')
    {
        char ch = tolower(str[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' ||
            ch == 'o' || ch == 'u')
        {
            count++;
        }
    }
}
```

```

        i++;
    }
    printf("Number of vowels = %d\n", count);
    return 0;
}

```

Input:

Compiler Design

Output:

Number of vowels = 5

Result:

Thus, the program was successfully implemented to count the number of vowels in a given sentence.

22: LEX Program to Print HTML Tags

Aim:

To write a LEX program to identify and print all HTML tags from an input HTML file.

Algorithm:

1. Start the LEX program.
2. Open the HTML input file.
3. Identify patterns starting with < and ending with >.
4. Print each identified HTML tag.
5. Ignore normal text.
6. Stop.

LEX Program:

```

%{
#include <stdio.h>
%}
%%

```

```

<[^>]+> { printf("HTML Tag: %s\n", yytext); }
.\n      { }
%%

int main()
{
    yylex();
    return 0;
}

```

Input: sample.html

```

<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>

```

Output:

```

HTML Tag: <html>
HTML Tag: <body>
HTML Tag: <h1>
HTML Tag: </h1>
HTML Tag: <p>
HTML Tag: </p>
HTML Tag: </body>
HTML Tag: </html>

```

Result:

Thus, the LEX program was successfully implemented to identify and print HTML tags.

23: LEX Program to Add Line Numbers

Aim:

To write a LEX program to add line numbers to each line of a C source program.

Algorithm:

1. Start the LEX program.
2. Initialize the line number as 1.
3. Print the first line number.
4. Read each character from the input.
5. Whenever a newline occurs, increment the line number.
6. Print the next line number.
7. Continue until the end of the file.

LEX Program:

```
%{  
#include <stdio.h>  
  
int line = 1;  
%}  
%%  
  
\n    {  
        printf("\n%d\t", ++line);  
    }  
  
.  
    {  
        ECHO;  
    }  
  
%%  
  
int main()  
{  
    printf("%d\t", line);  
    yylex();  
    return 0;  
}
```

Input:

```
#define PI 3.14  
#include<stdio.h>
```

```
#include<conio.h>

void main()
{
int a,b,c = 30;
printf("hello");
}
```

Output:

```
1  #define PI 3.14
2  #include<stdio.h>
3  #include<conio.h>
4  void main()
5  {
6  int a,b,c = 30;
7  printf("hello");
8  }
```

Result:

Thus, the LEX program was successfully implemented to add line numbers to a C source program.

24: Count and Eliminate Comment Lines

Aim:

To write a LEX program to count the number of comments in a C program, remove them, and write the remaining program into another file.

Algorithm:

1. Start the program.
2. Open the input C file.
3. Open an output file.
4. Identify single-line comments beginning with //.
5. Identify multi-line comments beginning with /* and ending with */.
6. Increment the comment counter.

7. Ignore the comments.
8. Write remaining content into the output file.
9. Display the number of comments.

LEX Program:

```
%{
#include <stdio.h>

int commentCount = 0;

FILE *out;

%}

%%

"//".*      { commentCount++; }

"/*"([^\*]|\\*+[^*/])*"*/" { commentCount++; }

.\n        { fprintf(out, "%s", yytext); }

%%

int main()
{
    out = fopen("output.c", "w");
    if (out == NULL)
    {
        printf("Cannot open output file\n");
        return 1;
    }

    yylex();
    fclose(out);
    printf("Number of comments = %d\n", commentCount);
    return 0;
}
```

Input:

```
#include<stdio.h>

int main()
```

```

{
int a,b,c; /* variable declaration */
printf("enter two numbers");
scanf("%d %d",&a,&b);
c=a+b;//adding two numbers
printf("sum is %d",c);
return 0;
}

```

Output:

Number of comments = 2

Output File:

```

#include<stdio.h>

int main()
{
int a,b,c;
printf("enter two numbers");
scanf("%d %d",&a,&b);
c=a+b;
printf("sum is %d",c);
return 0;
}

```

Result:

Thus, comments were successfully counted and eliminated from the C program.

25: Identify Capital Words

Aim:

To write a LEX program to identify capital words from the given input.

Algorithm:

1. Start.

2. Read the input.
3. Identify words containing only uppercase letters.
4. Print the identified capital words.
5. Ignore other words.
6. Stop.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
[A-Z]+ { printf("Capital Word: %s\n", yytext); }  
.\n { }  
%%  
int main()  
{  
    yylex();  
    return 0;  
}
```

Input:

HELLO this is INDIA and COMPUTER SCIENCE

Output:

Capital Word: HELLO

Capital Word: INDIA

Capital Word: COMPUTER

Capital Word: SCIENCE

Result:

Thus, the LEX program was successfully implemented to identify capital words.

26: Validate Email Address

Aim:

To write a LEX program to check whether a given email address is valid or not.

Algorithm:

1. Start.
2. Read the email address.
3. Check for a valid username.
4. Check for the @ symbol.
5. Check for a valid domain.
6. Check for a valid extension.
7. Display whether the email is valid or invalid.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}  
{  
    printf("Valid Email Address\n");  
}  
.|\\n  
{  
    printf("Invalid Email Address\n");  
}  
%%  
int main()  
{  
    yylex();  
    return 0;  
}
```

```
}
```

Input:

student@gmail.com

Output:

Valid Email Address

Result:

Thus, the LEX program was successfully implemented to validate an email address.

27: Convert abc to ABC

Aim:

To write a LEX program to convert every occurrence of the substring abc into ABC.

Algorithm:

1. Start.
2. Read the input string.
3. Search for the substring abc.
4. Replace every occurrence with ABC.
5. Print the modified string.
6. Stop.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
abc    { printf("ABC"); }  
.\n    { ECHO; }  
%%  
int main()  
{
```

```
yylex();  
return 0;  
}
```

Input:

abc is a simple abc example

Output:

ABC is a simple ABC example

Result:

Thus, the LEX program was successfully implemented to replace abc with ABC.

28: Validate Mobile Number

Aim:

To write a LEX program to validate a mobile number.

Algorithm:

1. Start.
2. Read the mobile number.
3. Check whether it contains exactly 10 digits.
4. Check whether the first digit is between 6 and 9.
5. If valid, display valid mobile number.
6. Otherwise, display invalid mobile number.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
[6-9][0-9]{9}  
{  
    printf("Valid Mobile Number\n");  
}
```

```

.\n
{
    printf("Invalid Mobile Number\n");
}
%%

int main()
{
    yylex();
    return 0;
}

```

Input:

9876543210

Output:

Valid Mobile Number

Result:

Thus, the LEX program was successfully implemented to validate a mobile number.

29: Token Separation Using LEX

Aim:

To implement a lexical analyzer using LEX to separate tokens from a given C program and display them with appropriate captions.

Algorithm:

1. Start.
2. Read the C source program.
3. Identify keywords.
4. Identify identifiers.
5. Identify constants.
6. Identify operators.

7. Identify delimiters.
8. Display each token with its corresponding type.

LEX Program:

```
%{  
#include <stdio.h>  
#include <string.h>  
int isKeyword(char *word)  
{  
    char *keywords[] = {  
        "int", "float", "char", "void", "return",  
        "if", "else", "while", "for"  
    };  
    for (int i = 0; i < 9; i++)  
    {  
        if (strcmp(word, keywords[i]) == 0)  
            return 1;  
    }  
    return 0;  
}  
%}  
%%  
[a-zA-Z_][a-zA-Z0-9_]*  
{  
    if (isKeyword(yytext))  
        printf("Keyword: %s\n", yytext);  
    else  
        printf("Identifier: %s\n", yytext);  
}  
[0-9]+  
{
```

```

    printf("Constant: %s\n", yytext);
}
[+\-*/=]
{
    printf("Operator: %s\n", yytext);
}
[(){};,]
{
    printf("Delimiter: %s\n", yytext);
}
[ \t\n]+  {}
.        {}
%%

```

```

int main()
{
    yylex();
    return 0;
}

```

Input:

```

#include<stdio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}

```

Output:

Keyword: void

Identifier: main

Delimiter: (

Delimiter:)

Delimiter: {
Keyword: int
Identifier: a
Delimiter: ,
Identifier: b
Delimiter: ,
Identifier: c
Operator: =
Constant: 30
Delimiter: ;
Identifier: printf
Delimiter: (
Identifier: hello
Delimiter:)
Delimiter: ;
Delimiter: }

Result:

Thus, the lexical analyzer was successfully implemented using LEX to separate tokens.

30: Count Number of Consonants

Aim:

To write a program to count the number of consonants in a given sentence.

Algorithm:

1. Start.
2. Read the input sentence.
3. Initialize consonant count to zero.
4. Check every alphabetic character.
5. If the character is not a vowel, increment the consonant count.

6. Display the result.

C Program:

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    char str[200];
    int i = 0, count = 0;
    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\0')
    {
        char ch = tolower(str[i]);
        if (isalpha(ch) &&
            ch != 'a' && ch != 'e' &&
            ch != 'i' && ch != 'o' &&
            ch != 'u')
        {
            count++;
        }
        i++;
    }
    printf("Number of consonants = %d\n", count);
    return 0;
}
```

Input:

Compiler Design

Output:

Number of consonants = 9

Result:

Thus, the program was successfully implemented to count consonants.

31: Separate Keywords and Identifiers

Aim:

To write a LEX program to differentiate and display keywords and identifiers.

Algorithm:

1. Start.
2. Read the source program.
3. Match reserved keywords.
4. Print them as keywords.
5. Match valid identifiers.
6. Print them as identifiers.
7. Ignore other tokens.

LEX Program:

```
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"double"|"void"|"return"|"if"|"else"|"while"|"for"
{
    printf("Keyword: %s\n", yytext);
}
[a-zA-Z_][a-zA-Z0-9_]*
{
    printf("Identifier: %s\n", yytext);
}
.\n { }
```

```
%%  
  
int main()  
{  
    yylex();  
    return 0;  
}
```

Input:

```
int main()  
{  
    int number = 10;  
    return number;  
}
```

Output:

Keyword: int

Identifier: main

Keyword: int

Identifier: number

Keyword: return

Identifier: number

Result:

Thus, keywords and identifiers were successfully separated.

32: Count Positive and Negative Numbers

Aim:

To write a LEX program to identify and count positive and negative numbers.

Algorithm:

1. Start.
2. Initialize positive and negative counters.

3. Read the input.
4. Identify positive numbers.
5. Increment the positive counter.
6. Identify negative numbers.
7. Increment the negative counter.
8. Display both counts.

LEX Program:

```
%{
#include <stdio.h>
int positive = 0;
int negative = 0;
%}
%%
-[0-9]+
{
    negative++;
    printf("Negative Number: %s\n", yytext);
}
[0-9]+
{
    positive++;
    printf("Positive Number: %s\n", yytext);
}
[ \t\n]+ { }
. { }
%%
int main()
{
    yylex();
}
```

```

printf("Positive Numbers = %d\n", positive);
printf("Negative Numbers = %d\n", negative);
return 0;
}

```

Input:

10 -20 30 -40 50

Output:

Positive Number: 10

Negative Number: -20

Positive Number: 30

Negative Number: -40

Positive Number: 50

Positive Numbers = 3

Negative Numbers = 2

Result:

Thus, positive and negative numbers were successfully identified and counted.

33: Validate URL

Aim:

To write a LEX program to validate a URL.

Algorithm:

1. Start.
2. Read the URL.
3. Check whether it begins with http:// or https://.
4. Check for a valid domain.
5. Check for a valid domain extension.
6. Display valid or invalid.

LEX Program:

```

%{
#include <stdio.h>

```

```

%}

%%

https?:/[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(/[a-zA-Z0-9./_-]*)?

{
    printf("Valid URL\n");
}

.\n

{
    printf("Invalid URL\n");
}

%%

int main()
{
    yylex();
    return 0;
}

```

Input:

https://www.google.com

Output:

Valid URL

Result:

Thus, the LEX program was successfully implemented to validate a URL.

34: Validate Date of Birth

Aim:

To write a LEX program to validate the Date of Birth.

Algorithm:

1. Start.
2. Read the DOB.

3. Check the format DD/MM/YYYY.
4. Validate day between 01 and 31.
5. Validate month between 01 and 12.
6. Display whether the DOB is valid or invalid.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}  
{  
    printf("Valid Date of Birth\n");  
}  
.|\\n  
{  
    printf("Invalid Date of Birth\n");  
}  
%%  
int main()  
{  
    yylex();  
    return 0;  
}
```

Input:

03/10/2006

Output:

Valid Date of Birth

Result:

Thus, the LEX program was successfully implemented to validate a Date of Birth.

35: Check Whether Input Is a Digit

Aim:

To write a LEX program to check whether the given input is a digit.

Algorithm:

1. Start.
2. Read the input character.
3. Check whether it is between 0 and 9.
4. If yes, display digit.
5. Otherwise, display not a digit.

LEX Program:

```
%{  
#include <stdio.h>  
%}  
%%  
[0-9]  
{  
    printf("It is a Digit\n");  
}  
.|\\n  
{  
    printf("It is not a Digit\n");  
}  
%%  
int main()  
{  
    yylex();  
    return 0;  
}
```

Input:

7

Output:

It is a Digit

Result:

Thus, the LEX program was successfully implemented to check whether the given input is a digit.

36: Basic Mathematical Operations

Aim:

To write a LEX program to implement basic arithmetic operations.

Algorithm:

1. Start.
2. Read two numbers and an operator.
3. Identify the operator.
4. Perform addition, subtraction, multiplication, or division.
5. Display the result.

C Program:

```
#include <stdio.h>

int main()
{
    float a, b;
    char op;
    printf("Enter expression (example 10 + 20): ");
    scanf("%f %c %f", &a, &op, &b);
    switch (op)
    {
        case '+':
            printf("Result = %.2f", a + b);
            break;
```



```

    case '-':
        printf("Result = %.2f", a - b);
        break;
    case '*':
        printf("Result = %.2f", a * b);
        break;
    case '/':
        if (b != 0)
            printf("Result = %.2f", a / b);
        else
            printf("Division by zero is not possible");
        break;
    default:
        printf("Invalid Operator");
}
return 0;
}

```

Input:

10 + 20

Output:

Result = 30.00

Result:

Thus, the program was successfully implemented to perform basic mathematical operations.

37: Frequency of a Given Word

Aim:

To write a LEX program to count the frequency of a given word in a sentence.

Algorithm:

1. Start.
2. Read the input sentence.
3. Define the target word.
4. Compare every word with the target.
5. Increment the counter when a match occurs.
6. Display the frequency.

LEX Program:

```
%{  
#include <stdio.h>  
  
int count = 0;  
  
%  
%%  
  
compiler  
{  
    count++;  
}  
[a-zA-Z]+ { }  
.\n { }  
%%  
  
int main()  
{  
    yylex();  
    printf("Frequency of word compiler = %d\n", count);  
    return 0;  
}
```

Input:

compiler design compiler construction compiler

Output:

Frequency of word compiler = 3

Result:

Thus, the LEX program was successfully implemented to count the frequency of a word.

38: Length of the Longest Word**Aim:**

To write a LEX program to find the length of the longest word in a given input.

Algorithm:

1. Start.
2. Initialize maximum length to zero.
3. Read each word.
4. Find the length of the word.
5. Compare it with the maximum length.
6. Update the maximum if necessary.
7. Display the longest length.

LEX Program:

```
%{  
#include <stdio.h>  
  
int max = 0;  
%}  
%%  
[a-zA-Z]+  
{  
    if (yyleng > max)  
        max = yylen;  
}  
.\n { }  
%%  
  
int main()  
{
```

```

yylex();
printf("Length of longest word = %d\n", max);
return 0;
}

```

Input:

C programming lexical analyzer compiler

Output:

Length of longest word = 11

Result:

Thus, the LEX program was successfully implemented to find the length of the longest word.

39: Replace a Word in a File

Aim:

To write a LEX program to replace a specified word with another word in a file.

Algorithm:

1. Start.
2. Read the input file.
3. Search for the specified word.
4. Replace the word.
5. Print the modified content.
6. Stop.

LEX Program:

```

%{
#include <stdio.h>
%}
%%
hello
{

```

```

    printf("welcome");
}
.\n
{
    ECHO;
}
%%

int main()
{
    yylex();
    return 0;
}

```

Input:

hello students

hello everyone

Output:

welcome students

welcome everyone

Result:

Thus, the LEX program was successfully implemented to replace a word with another word.

40: FLEX Program for Token Separation

Aim:

To write a FLEX program to separate tokens from a given C program and display each token with an appropriate caption.

Algorithm:

1. Start the FLEX program.
2. Read the C source program.
3. Identify keywords.

4. Identify identifiers.
5. Identify numeric constants.
6. Identify operators.
7. Identify delimiters.
8. Ignore spaces and comments.
9. Display every token with its category.

FLEX Program:

```
%{
#include <stdio.h>
#include <string.h>
int isKeyword(char *word)
{
    char *keywords[] = {
        "int", "float", "char", "double",
        "void", "return", "if", "else",
        "while", "for"
    };
    int i;
    for (i = 0; i < 10; i++)
    {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}
}%
%%
"//".*      { }
"/*"([^\*]|\\*+[^*/])**"/" { }
```

```

[a-zA-Z_][a-zA-Z0-9_]*
{
    if (isKeyword(yytext))
        printf("KEYWORD   : %s\n", yytext);
    else
        printf("IDENTIFIER : %s\n", yytext);
}
[0-9]+(\.[0-9]+)?
{
    printf("CONSTANT   : %s\n", yytext);
}
"=="|"!="|"<="|">="|"="|"+"|"-|"*"|"|"
{
    printf("OPERATOR   : %s\n", yytext);
}
"(")"|"{"|"}"|"|";"
{
    printf("DELIMITER  : %s\n", yytext);
}
[ \t\n]+    { }
.           { }
%%

int main()
{
    yylex();
    return 0;
}

```

Input:

```
#include<stdio.h>
```

```
void main()
{
    int a,b,c = 30;
    printf("hello");
}
```

Output:

KEYWORD : void

IDENTIFIER : main

DELIMITER : (

DELIMITER :)

DELIMITER : {

KEYWORD : int

IDENTIFIER : a

DELIMITER : ,

IDENTIFIER : b

DELIMITER : ,

IDENTIFIER : c

OPERATOR : =

CONSTANT : 30

DELIMITER : ;

IDENTIFIER : printf

DELIMITER : (

IDENTIFIER : hello

DELIMITER :)

DELIMITER : ;

DELIMITER : }

Result:

Thus, the FLEX program was successfully implemented to separate tokens from a C source program and display them with appropriate captions.